

Escuela de Ingeniería y Tecnologías

Proyecto de Introducción a la Ingeniería

Avances del Proyecto

Profesor: Antonio Martinez Torteya.

Esaúl Alejandro Villarreal Ábrego 646239
Ángel Gabriel Romero Patena 694550

“Doy mi palabra que he realizado esta actividad con integridad académica.”

- Esaúl Alejandro Villarreal Ábrego
- Ángel Gabriel Romero Patena

Resumen del Proyecto

Consiste en replicar de manera inexacta el método de transmisión y recepción laser que están probando la NASA y la ESA para mejorar la comunicación con naves espaciales lejos de la Tierra. Para conseguir esto, se planea crear tres sistemas:

1. El primero, con base en inputs del usuario, manda señales infrarrojas al segundo con comandos específicos.
    a. También recibe información del tercer sistema a través del segundo.
2. Este sistema recibe e interpreta las señales infrarrojas enviadas por el primer sistema (el equivalente a un satélite), mandando los comandos requeridos al tercero mediante Bluetooth.
    a. Una vez que el rover manda la información recolectada, el segundo sistema envía la información de regreso al primero.
3. El tercero consiste de un rover que, al recibir las señales Bluetooth, ejecuta los comandos enviados y regresa la información recolectada a través de sus sensores.

Los objetivos, y bases de calificación, son:

a) Conectar los tres sistemas de tal manera que sea posible la comunicación bidireccional, clara y ordenada (mandando comandos y recibiendo información recolectada por el rover).
b) Crear sistemas robustos capaces de conectarse automáticamente entre sí al ser encendidos. En caso de perder comunicación, deberán restablecerla automáticamente e informar al usuario de la situación y del progreso realizado en resolver el problema.

Avances de Medio Término

Durante la sesión donde se mostraron avances del proyecto, se demostró un cumplimiento parcial del **Objetivo A** al establecer una comunicación unidireccional entre los sistemas, permitiendo controlar el tercer sistema a través de un joystick conectado al primer sistema. Sin embargo, la comunicación entre el segundo y el tercer sistema dependía de una computadora dado a que no se tenía un módulo bluetooth capaz de conectarse con el rover. Este método, aunque exitoso, no es práctico para el proyecto final.

Progreso

Hardware

Desde esa sesión, se ha reemplazado la computadora y el Arduino del segundo sistema por una unidad ESP32-WROOM-E32 (ver **Figura 1**), la cual incluye comunicación bluetooth incluída y capaz de comunicarse con el rover. Esto facilita el proceso de comunicación al eliminar el uso de una computadora. Las conexiones fueron realizadas con apoyo de Gemini, la IA de Google (Google, 2026).

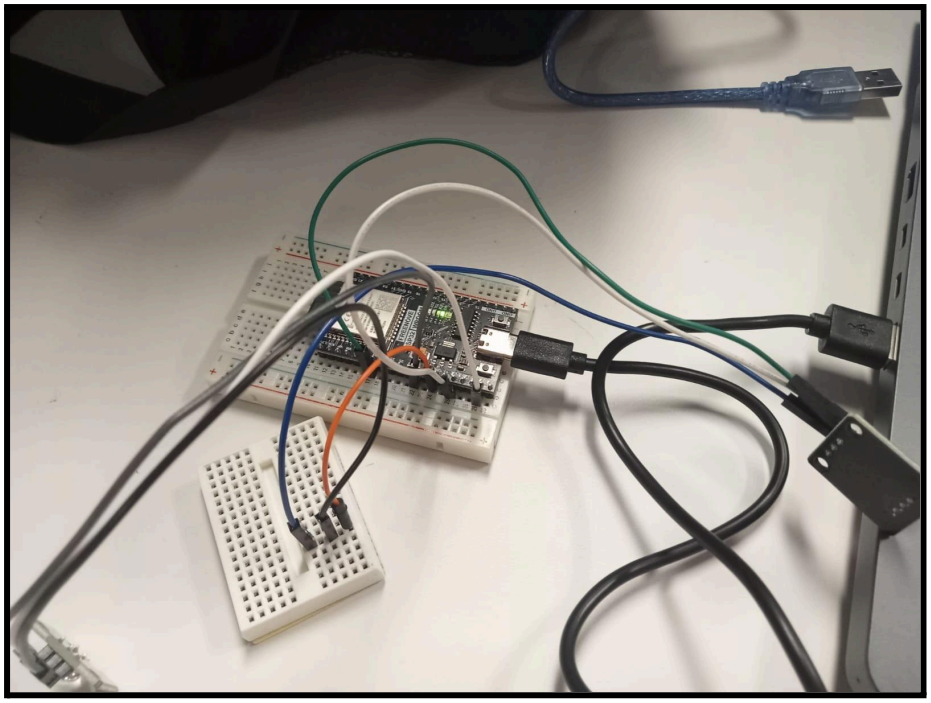


Figura 1: Se ha reemplazado al Arduino y a la computadora previamente usados en el sistema dos por una unidad ESP32 (la computadora visible en la parte derecha de la imagen solo da poder y posibilidad de realizar debugging).

Así mismo, se le ha agregado un emisor IR al segundo sistema y un receptor IR al primero para, eventualmente, establecer una comunicación bidireccional y cumplir el **Objetivo A** (ver **Figura 2**).



Figura 2: Se ha agregado otro par de emisor-receptor IR al proyecto para permitir la comunicación bidireccional.

Software - Vehículo

Se modificaron los códigos que administran el segundo y tercer sistema (ver **Figura 3** y **Figura 4** respectivamente) para que fueran compatibles con el ESP32-WROOM-E32. Dado a que la funcionalidad bluetooth del ESP32 es programado de manera distinta al Arduino, se hizo uso de la Inteligencia Artificial Gemini para adaptar tanto el software del segundo sistema como el software del carro para enviar y recibir comandos correctamente (Google, 2026).

Aunque todavía no se comprende en su totalidad los comandos usados en el código, se está analizando activamente para averiguarlo e integrarlo en nuestra explicación técnica y en nuestro conocimiento. Lo que sí se sabe es que se utiliza:

- Un MAC Address para saber a qué módulo conectarse.
- UUIDs (Universally Unique Identifier) para identificar precisamente los servicios disponibles para Bluetooth (ej. Escribir al módulo) dentro de un GATT (General Attribute Profile) que contiene múltiples servicios, datos y descriptores de datos (RF Wireless Expert Team, 2026).



```cpp
#include <IRremote.hpp>
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEClient.h>

// --- TUS PINES EXACTOS ---
const int IR_SEND_PIN = 14;
const int IR_RECEIVE_PIN = 32;

// --- LA MAC EXACTA DE TU ROVER ---
String targetMacAddress = "48:87:2D:12:13:C5";

// --- UUIDs SERIALES DEL HC-08 ---
static BLEUUID serviceUUID((uint16_t)0xFFE0);
static BLEUUID charUUID((uint16_t)0xFFE2);

boolean connected = false;
BLEClient* pClient;
BLERemoteCharacteristic* pRemoteCharacteristic;

bool connectToRover() {
    Serial.print("Intentando conectar directamente a la MAC: ");
    Serial.println(targetMacAddress.c_str());

    // Crear el cliente BLE
    pClient = BLEDevice::createClient();

    // Conexión directa por dirección MAC (B Anula la necesidad de escanear)
    if (!pClient->connect(BLEAddress(targetMacAddress.c_str()))) {
      Serial.println(" -> [FALLO] El HC-08 rechazó la conexión o está fuera de rango.");
      delete pClient;
      return false;
    }
    Serial.println(" -> [OK] ¡Conexión física establecida con el rover!");

    // Buscar el servicio serial 0xFFE0
    BLERemoteService* pRemoteService = pClient->getService(serviceUUID);
    if (pRemoteService == nullptr) {
      Serial.println(" -> [FALLO] No se encontró el servicio UUID 0xFFE0.");
      pClient->disconnect();
      delete pClient;
      return false;
    }
    Serial.println(" -> [OK] Servicio 0xFFE0 encontrado.");

    // Buscar la característica 0xFFE1 (para enviar datos)
    pRemoteCharacteristic = pRemoteService->getCharacteristic(charUUID);
    if (pRemoteCharacteristic == nullptr) {
      Serial.println(" -> [FALLO] No se encontró la característica UUID 0xFFE1.");
      pClient->disconnect();
      delete pClient;
      return false;
    }

    Serial.println("¡ENLACE BLE COMPLETAMENTE LISTO!");
    return true;
}
```

```cpp
void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("\n--- Sistema Iniciado: Conexión Directa por MAC ---");

  // Iniciar periféricos IR
  IrReceiver.begin(IR_RECEIVE_PIN, ENABLE_LED_FEEDBACK);
  IrSender.begin(IR_SEND_PIN);

  // Inicializar antena Bluetooth
  BLEDevice::init("ESP32_Maestro");
}

void loop() {
  // Si no estamos conectados, intentar la conexión de forma continua con pausas
  if (!connected) {
    if (connectToRover()) {
      connected = true;
    } else {
      Serial.println("Reintentando conexión en 3 segundos...\n");
      delay(3000);
    }
  }

  // Si ya hay enlace activo, escuchar el control remoto IR y mandarlo al rover
  if (connected) {
    // Verificar si el cliente sigue conectado físicamente
    if (!pClient->isConnected()) {
      Serial.println("\n[ALERTA] Se perdió la conexión con el rover. Reiniciando bucle de enlace...");
      connected = false;
      return;
    }

    if (IrReceiver.decode()) {
      uint32_t command = IrReceiver.decodedIRData.command;
      String dataToSend = "";

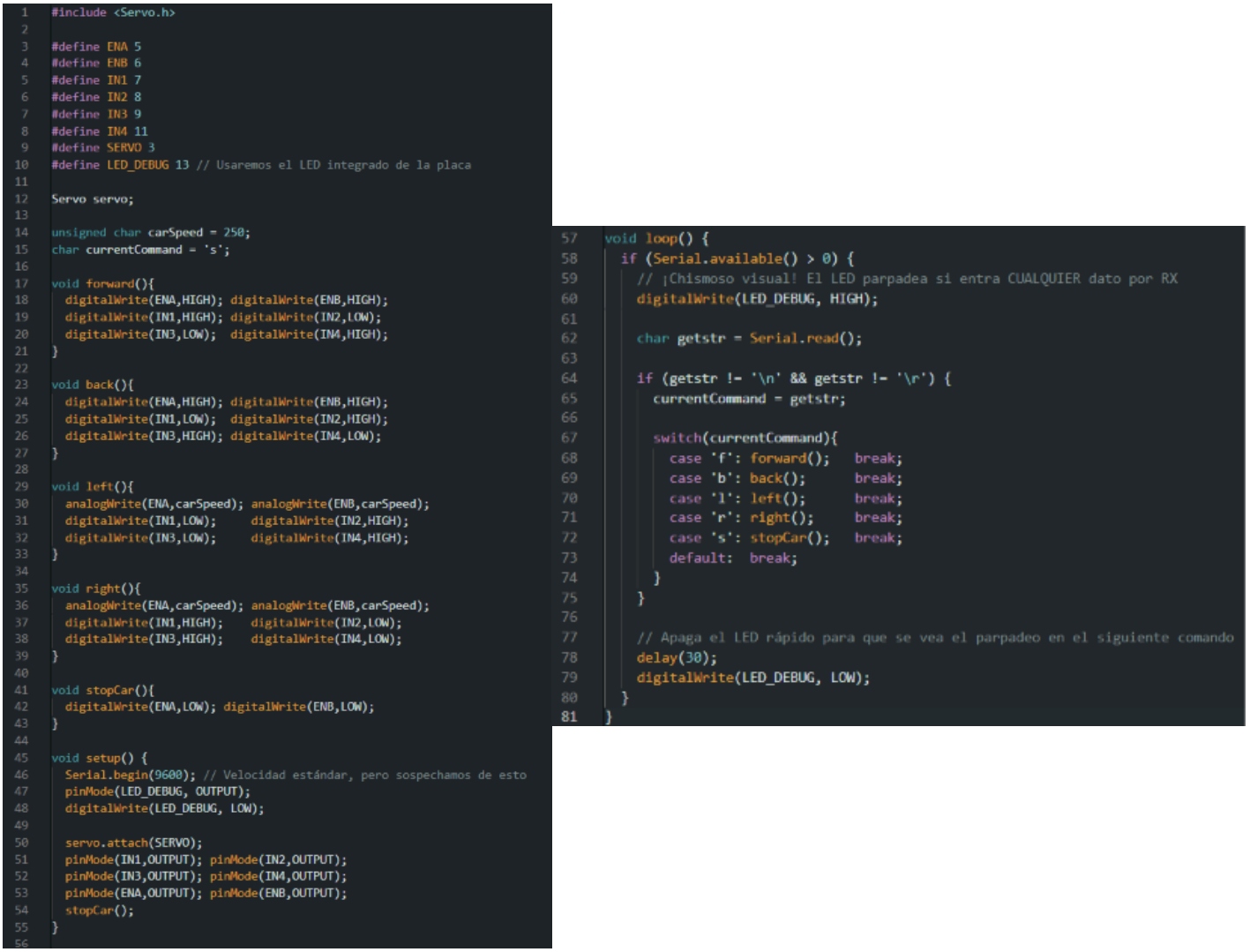
      // Agregamos \n para obligar al HC-08 a vaciar su buffer
      if (command == 0x66) dataToSend = "f\n";      // Adelante
      else if (command == 0x62) dataToSend = "b\n"; // Atrás
      else if (command == 0x6C) dataToSend = "l\n"; // Izquierda
      else if (command == 0x72) dataToSend = "r\n"; // Derecha
      else dataToSend = "s\n";                      // Detener

      Serial.print("IR detectado (0x");
      Serial.print(command, HEX);
      Serial.print(") -> Enviando por Bluetooth: ");
      Serial.println(dataToSend);

      // Enviar datos al HC-08
      if (pRemoteCharacteristic != nullptr) {
        pRemoteCharacteristic->writeValue((uint8_t*)dataToSend.c_str(), dataToSend.length(), false);
      }

      IrReceiver.resume();
    }
  }
}
```

Figura 3: Código que controla al segundo sistema. Se utilizó la IA de Google, Gemini, para adaptar el código previo (que usaba Arduino y un módulo Bluetooth separado) para que fuera compatible con el ESP32 (Google, 2026).



```ino
#include <Servo.h>

#define ENA 5
#define ENB 6
#define IN1 7
#define IN2 8
#define IN3 9
#define IN4 11
#define SERVO 3
#define LED_DEBUG 13 // Usaremos el LED integrado de la placa

Servo servo;

unsigned char carSpeed = 250;
char currentCommand = 's';

void forward(){
  digitalWrite(ENA,HIGH); digitalWrite(ENB,HIGH);
  digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
  digitalWrite(IN3,LOW);  digitalWrite(IN4,HIGH);
}

void back(){
  digitalWrite(ENA,HIGH); digitalWrite(ENB,HIGH);
  digitalWrite(IN1,LOW);  digitalWrite(IN2,HIGH);
  digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
}

void left(){
  analogWrite(ENA,carSpeed); analogWrite(ENB,carSpeed);
  digitalWrite(IN1,LOW);     digitalWrite(IN2,HIGH);
  digitalWrite(IN3,LOW);     digitalWrite(IN4,HIGH);
}

void right(){
  analogWrite(ENA,carSpeed); analogWrite(ENB,carSpeed);
  digitalWrite(IN1,HIGH);    digitalWrite(IN2,LOW);
  digitalWrite(IN3,HIGH);    digitalWrite(IN4,LOW);
}

void stopCar(){
  digitalWrite(ENA,LOW); digitalWrite(ENB,LOW);
}

void setup() {
  Serial.begin(9600); // Velocidad estándar, pero sospechamos de esto
  pinMode(LED_DEBUG, OUTPUT);
  digitalWrite(LED_DEBUG, LOW);

  servo.attach(SERVO);
  pinMode(IN1,OUTPUT); pinMode(IN2,OUTPUT);
  pinMode(IN3,OUTPUT); pinMode(IN4,OUTPUT);
  pinMode(ENA,OUTPUT); pinMode(ENB,OUTPUT);
  stopCar();
}

void loop() {
  if (Serial.available() > 0) {
    // ¡Chismoso visual! El LED parpadea si entra CUALQUIER dato por RX
    digitalWrite(LED_DEBUG, HIGH);

    char getstr = Serial.read();

    if (getstr != '\n' && getstr != '\r') {
      currentCommand = getstr;

      switch(currentCommand){
        case 'f': forward();   break;
        case 'b': back();      break;
        case 'l': left();      break;
        case 'r': right();     break;
        case 's': stopCar();   break;
        default:  break;
      }
    }

    // Apaga el LED rápido para que se vea el parpadeo en el siguiente comando
    delay(30);
    digitalWrite(LED_DEBUG, LOW);
  }
}
```

Figura 4: Código usado para el rover. Se usó la IA de Google, Gemini, para mejorar la claridad del código y asegurar que los comandos enviados por el ESP32 fueran procesados correctamente. También se usó Gemini para agregar código para controlar el LED incluído en el Arduino, de manera que se pudiera saber si el rover estaba recibiendo comandos (Google, 2026).

Software - Aplicación

Para mandar comandos al rover, se usará una aplicación de computadora. Por el momento, se creó un programa base como referencia para ir agregando funcionalidad conforme se va completando y probando el proyecto. El código y la aplicación base se pueden observar en el repositorio donde viene este proyecto.

Resultados

Al intentar conectar los sistemas entre sí, se descubrió que se podía establecer una conexión exitosa con el rover. Sin embargo, este no respondía a ningún comando que era enviado. Tras múltiples intentos y modificaciones, hechos con apoyo de Gemini, se descubrió que se estaba usando el UUID incorrecto para identificar el servicio de escritura al módulo Bluetooth del rover. Esto pudo identificarse dado a experiencia previa trabajando con la computadora para enviar comandos al rover. Con apoyo de Gemini, se había identificado que el UUID correcto es 0xFFE2. En este caso, se estaba usando 0xFFE1. Tras cambiar este valor, el rover comenzó a recibir y ejecutar los comandos enviados. Un video del rover ejecutando los comandos enviados se puede ver en el repositorio Avance_Proyecto (donde se encuentra este documento).

Conclusiones

A futuro, el siguiente paso es poder hacer que el rover envíe información de regreso, en vez de solo recibir comandos. Ahora que se tiene el ESP32, esta comunicación podrá realizarse con mayor facilidad y practicidad. Además, se empezará a ensamblar el hardware y el software necesarios para establecer y mantener una comunicación autónoma entre los sistemas una vez que sean encendidos (cumpliendo el **Objetivo B**). Para el siguiente avance se procurará tener:

1. Diseño y prototipos de hardware.
2. Software para conexión automática.
3. Capacidad de comunicación bidireccional.
4. Entendimiento profundo del funcionamiento de Bluetooth.

Referencias

Google. (2026, 4 de noviembre). *Rama-Inicio de asesoría mecatrónica* [Chat de IA generativa]. Gemini 3.1. https://share.gemini.google/2zXU5O2xoeXA

RF Wireless Expert Team. (2026, 25 de agosto). *UUID and its usage in BLE (Bluetooth Low Energy)*. RF Wireless World. Recuperado el 5 de noviembre del 2026 de: https://www.rfwireless-world.com/terminology/ble-uuid.